

falco_security_prometheus_rules.yaml

Enterprise DevSecOps Security Lab

Prepared By	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Document Type	Prometheus Alert Rules Design and YAML Artifact Review
Environment	Private Ubuntu VM, Kind Kubernetes, Falco, Prometheus, Grafana
Classification	Portfolio / Private Lab Evidence

Document Intent

This professional portfolio artifact describes the design, implementation, validation, and operational usage of runtime detection, alerting, dashboards, or response artifacts created inside the private Enterprise DevSecOps lab.

Coverage Area	Included in this document
Runtime detection	Falco event logic, validation evidence, and triage workflow
Security observability	Prometheus metrics, Grafana dashboarding, alert health checks
Response readiness	Analyst actions, containment commands, retest criteria, evidence handling
Portfolio value	Professional documentation showing detection engineering, SOC process, and cloud-native security maturity

Lab boundary: no external systems were targeted. All attack simulations, detections, and response workflows were performed inside an isolated private VM and Kind Kubernetes environment.

Executive Summary

This document explains the PrometheusRule YAML artifact used to alert on Falco runtime detections and detection-health risks in the Enterprise DevSecOps lab. The rules convert raw runtime metrics into actionable security alerts.

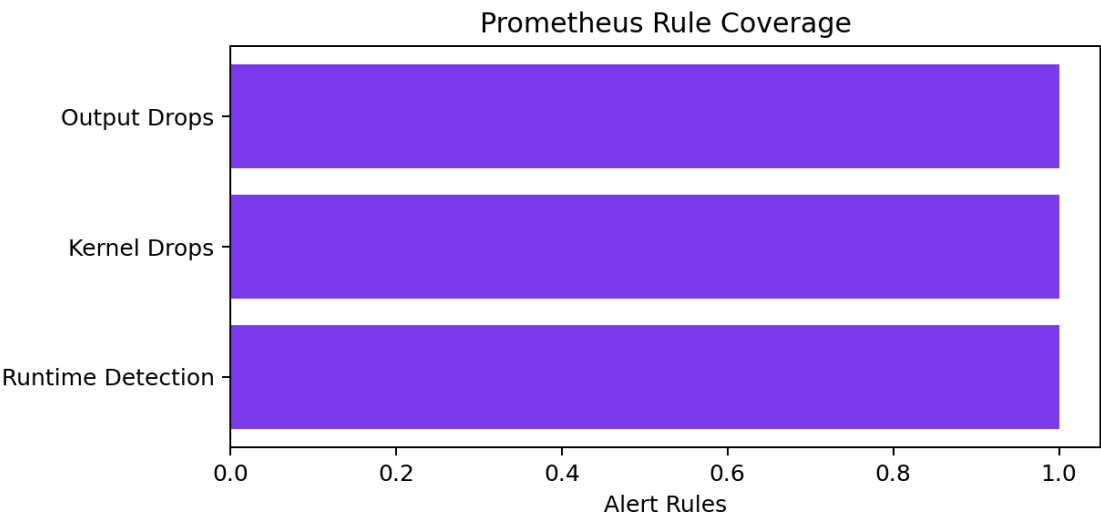
The file contains three alerting rules: FalcoRuntimeDetection, FalcoKernelEventDrops, and FalcoOutputQueueDrops. Together, they cover both security detections and detection pipeline reliability.

PrometheusRule Artifact Overview

Field	Value
API version	monitoring.coreos.com/v1
Kind	PrometheusRule
Namespace	monitoring
Rule group	falco.security.rules
Primary metric	falcosecurity_falco_rules_matches_total
Health metrics	falcosecurity_scap_n_drops_total; falcosecurity_falco_outputs_queue_num_drops_total

Alert Rule Inventory

Alert	Severity	Expression Purpose	Operational Meaning
FalcoRuntimeDetection	warning	increase(rule matches over 5 minutes) > 0	A Falco detection fired and requires triage
FalcoKernelEventDrops	critical	increase(kernel event drops over 5 minutes) > 0	Falco may be missing events
FalcoOutputQueueDrops	critical	increase(output queue drops over 5 minutes) > 0	Falco may be failing to deliver alerts



Prometheus security rule coverage implemented for Falco detections and telemetry health.

Rule Design Rationale

The rules use increase() over short windows to detect new events without depending on a continuously high absolute counter value. The runtime detection rule groups by rule_name, priority, and source so the resulting alert remains actionable. Health rules focus on dropped events because missed telemetry can undermine runtime detection integrity.

```
# Runtime detection alert logic
sum by (rule_name, priority, source) (
  increase(falcosecurity_falco_rules_matches_total[5m])
) > 0

# Telemetry health alert logic
sum(increase(falcosecurity_scap_n_drops_total[5m])) > 0
sum(increase(falcosecurity_falco_outputs_queue_num_drops_total[5m])) > 0
```

Full YAML Artifact

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: falco-security-rules
  namespace: monitoring
  labels:
    release: monitoring
spec:
  groups:
    - name: falco.security.rules
      rules:
        - alert: FalcoRuntimeDetection
          expr: |
            sum by (rule_name, priority, source) (
              increase(falcosecurity_falco_rules_matches_total[5m])
            ) > 0
          for: 0m
          labels:
            severity: warning
            category: runtime-security
          annotations:
            summary: "Falco runtime detection triggered"
            description: "Falco rule {{ $labels.rule_name }} triggered with priority {{
              $labels.priority }} from source {{ $labels.source }}."

        - alert: FalcoKernelEventDrops
          expr: |
            sum(increase(falcosecurity_scap_n_drops_total[5m])) > 0
          for: 1m
          labels:
            severity: critical
            category: runtime-security
          annotations:
            summary: "Falco kernel event drops detected"
            description: "Falco is dropping kernel events. Detection quality may be affected."

        - alert: FalcoOutputQueueDrops
          expr: |
            sum(increase(falcosecurity_falco_outputs_queue_num_drops_total[5m])) > 0
          for: 1m
          labels:
            severity: critical
            category: runtime-security
          annotations:
            summary: "Falco output queue drops detected"
            description: "Falco output queue is dropping events."
```

Alert Routing Recommendations

Alert	Recommended Route	Why
FalcoRuntimeDetection	Security triage queue / Slack channel	Requires analyst validation and workload owner review
FalcoKernelEventDrops	Platform/SRE and security on-call	Detection quality degradation can hide attacks
FalcoOutputQueueDrops	Platform/SRE and security on-call	Alert delivery failure can cause blind spots

Validation Procedure

```
# Apply rule
kubectl apply -f monitoring/rules/falco-security-prometheus-rules.yaml

# Confirm rule object exists
kubectl get prometheusrule -n monitoring | grep falco

# Generate safe Falco event
kubectl -n observability-test exec test-shell -- sh -c 'cat /etc/shadow >/dev/null 2>&1 || true'

# Query Prometheus
curl -sG "http://localhost:9090/api/v1/query" --data-urlencode 'query=sum(increase(falcosecurity_falco_rules_matches_total[10m]))'

# Review rules in Prometheus UI
http://localhost:9090 -> Status -> Rules
```

```
UID: f5abdf43-08de-45de-8efb-7b47b572d85d
Spec:
  Groups:
    Name: falco.security.rules
    Rules:
      Alert: FalcoRuntimeDetection
      Annotations:
        Description: Falco rule {{ $labels.rule_name }} triggered with priority {{ $labels.priority }} from source {{ $labels.source }}.
        Summary: Falco runtime detection triggered
        Expr: sum by (rule_name, priority, source) (
          increase(falcosecurity_falco_rules_matches_total[5m])
        ) > 0
      For: 0m
      Labels:
        Category: runtime-security
        Severity: warning
      Alert: FalcoKernelEventDrops
      Annotations:
        Description: Falco is dropping kernel events. Detection quality may be affected.
        Summary: Falco kernel event drops detected
        Expr: sum(increase(falcosecurity_scap_n_drops_total[5m])) > 0
      For: 1m
      Labels:
        Category: runtime-security
        Severity: critical
      Alert: FalcoOutputQueueDrops
      Annotations:
        Description: Falco output queue is dropping events.
        Summary: Falco output queue drops detected
        Expr: sum(increase(falcosecurity_falco_outputs_queue_num_drops_total[5m])) > 0
      For: 1m
      Labels:
        Category: runtime-security
        Severity: critical
      Events:
        Severity: <none>
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Evidence: Prometheus rules for Falco were visible in Prometheus after applying the YAML artifact.

```
Prometheus query: Falco detections in last 10 minutes
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {},
        "value": [
          1781610728.433,
          "8.42152541897089"
        ]
      }
    ]
  }
}
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Evidence: Prometheus query result validated Falco runtime detection metrics.

Tuning and Production Hardening

- Add namespace and workload owner labels to annotations where available.
- Change FalcoRuntimeDetection routing by priority so Critical alerts page faster than Warning alerts.

- Add `runbook_url` annotation for each alert to reduce response time.
- Define inhibition rules if a broader `FalcoDown` or `PrometheusDown` alert is firing.
- Baseline event drop thresholds in a realistic workload environment before production rollout.

Acceptance Criteria

Criterion	Expected Result
YAML applies without errors	PrometheusRule object created in monitoring namespace
Falco event generated	FalcoRuntimeDetection alert query returns > 0
No telemetry drop during normal lab workload	Drop metrics remain zero
Dashboard alignment	Grafana panels use the same metric names as alert rules

References

Falco rules basic elements - <https://falco.org/docs/concepts/rules/basic-elements/>

Falco supported fields for rule conditions and outputs - <https://falco.org/docs/reference/rules/supported-fields/>

Falco metrics - <https://falco.org/docs/concepts/metrics/>

Falco alert forwarding and JSON outputs - <https://falco.org/docs/concepts/outputs/forwarding/>

Prometheus alerting rules - https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

Grafana dashboard JSON model -

<https://grafana.com/docs/grafana/latest/visualizations/dashboards/build-dashboards/view-dashboard-json-model/>

Grafana dashboard import workflow - <https://grafana.com/docs/grafana/latest/dashboards/build-dashboards/import-dashboards/>